

YOLOv11 Object Detection

YOLOv11 Object Detection

1. Course Content
2. Launch Case
3. Core Code Analysis

1. Course Content

- Run examples, use YOLOv11 quantized model for accelerated object detection

2. Launch Case

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

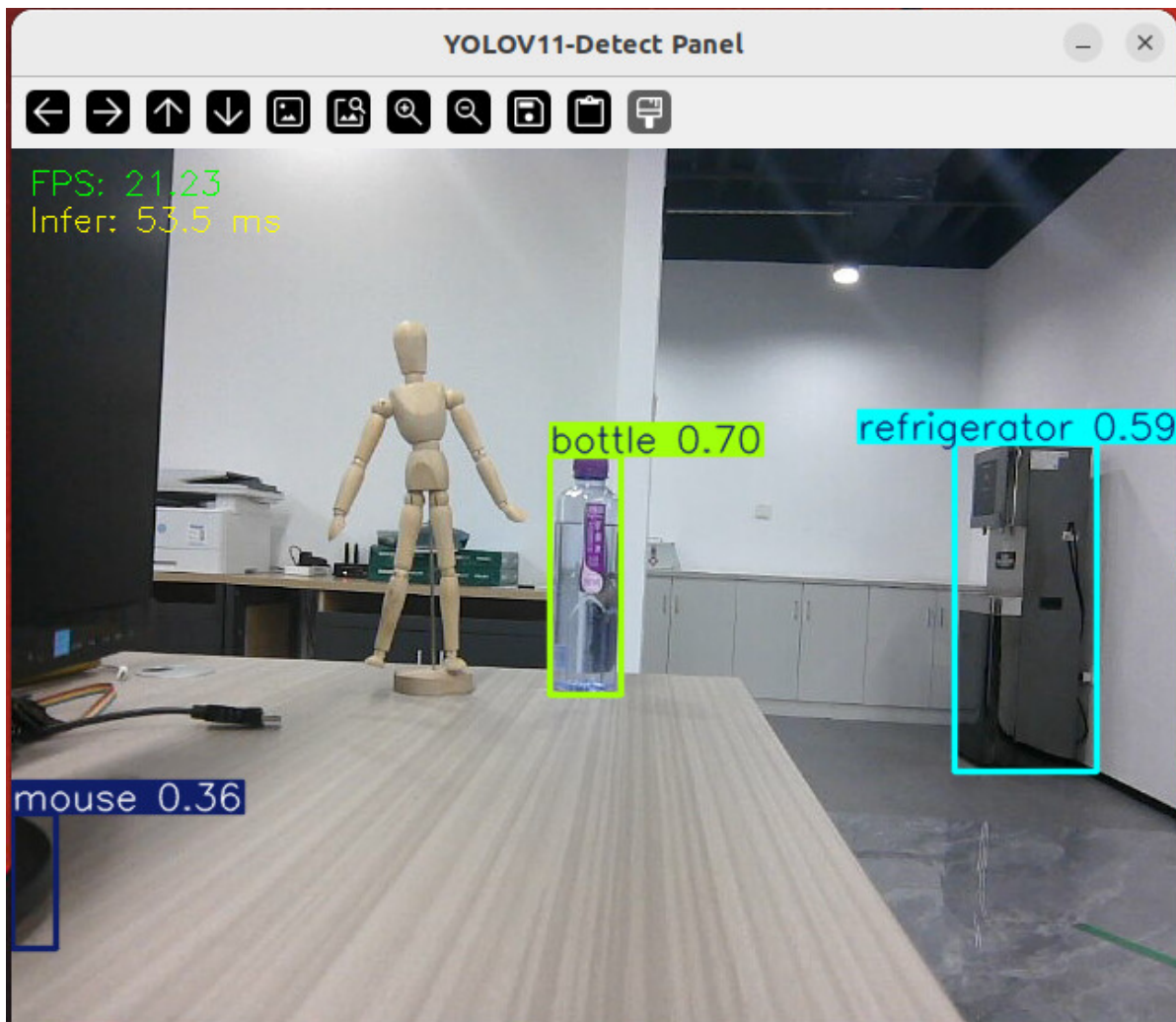
Open a terminal inside the container,

```
exec_m3
```

- Run command in terminal:

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=yolov11_detect
```

After the program runs, a `YOLOv11-Detect Panel` window opens, performing object detection through the YOLOv11 model, where FPS is the average frame rate per second, and Infer is the inference time for each frame image



- The YOLOv11 object detection model used here is a pre-trained model on the COCO8 dataset, recognizable objects are as follows:

```
names:
  0: person
  1: bicycle
  2: car
  3: motorcycle
  4: airplane
  5: bus
  6: train
  7: truck
  8: boat
  9: traffic light
  10: fire hydrant
  11: stop sign
  12: parking meter
  13: bench
  14: bird
  15: cat
  16: dog
  17: horse
  18: sheep
  19: cow
  20: elephant
  21: bear
```

22: zebra
23: giraffe
24: backpack
25: umbrella
26: handbag
27: tie
28: suitcase
29: frisbee
30: skis
31: snowboard
32: sports ball
33: kite
34: baseball bat
35: baseball glove
36: skateboard
37: surfboard
38: tennis racket
39: bottle
40: wine glass
41: cup
42: fork
43: knife
44: spoon
45: bowl
46: banana
47: apple
48: sandwich
49: orange
50: broccoli
51: carrot
52: hot dog
53: pizza
54: donut
55: cake
56: chair
57: couch
58: potted plant
59: bed
60: dining table
61: toilet
62: tv
63: laptop
64: mouse
65: remote
66: keyboard
67: cell phone
68: microwave
69: oven
70: toaster
71: sink
72: refrigerator
73: book
74: clock
75: vase
76: scissors
77: teddy bear
78: hair drier
79: toothbrush

- To get the highest frame rate and inference speed, the YOLOv11 model used in this case has been optimized for different platforms (OrinNano, OrinNX, Raspberry Pi, RDKx5) in model format, model paths are as follows:

```
~/MODELS/yolov11/
```

3. Core Code Analysis

Source code paths are as follows:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/yolov11/yolov11_detect.py
```

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/yolov11/yolov11_infer.py
```

- Class in yolov11_infer.py: YOLO_Detect is a core functionality program that encapsulates model loading and inference

```
class YOLO_Detect(YOLO_MODEL):
    """YOLOv11 object detection"""
    def __init__(self, model_path: str):
        self.model_path = model_path

    def init_yolo_engine(self):
        "Initialization of model"
        try:
            self.yolo_model = YOLO(self.model_path, task="detect")
        except:
            return False
        return True

    def get_infer_result(self, source, stream: bool = True):
        '''Obtain the reasoning result of a single frame image'''
        result = self.yolo_model.predict(source,
                                         stream=stream,
                                         half=True,
                                         max_det=3,
                                         verbose=False)

        return result
```

- The get_RGBImageCallBack callback function in yolov11_detect.py program is the core program for displaying detection frames, with the following functions:
- Subscribe to camera topic
- Use yolo_model.get_infer_result function method to perform inference on single frame images of camera feed and draw on the frame
- Use cv2.imshow function to display image frame rate and inference time

```

def get_RGBImageCallback(self, rgb_msg):
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(rgb_msg,
desired_encoding='bgr8')
    infer_start = time.time()
    results= self.yolo_model.get_infer_result(rgb_image,True)
    for res in results:
        if res.bboxes is not None:
            annotated_frame = res.plot()
    infer_end = time.time()
    self.frame_count += 1
    curr_time = time.time()
    time_diff = curr_time - self.prev_time
    if time_diff >= 1.0:
        self.fps = self.frame_count / time_diff
        self.frame_count = 0
        self.prev_time = curr_time
    cv2.putText(annotated_frame, f"FPS: {self.fps:.2f}", (10,
25), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 1,)
    infer_time = (infer_end - infer_start) * 1000
    cv2.putText(annotated_frame, f"Infer: {infer_time:.1f} ms", (10,
45), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)
    cv2.imshow("YOLOV11-Detect Panel", annotated_frame)
    cv2.waitKey(1)

```